

Active Data Collection with Cost-Aware Transport Heuristics for Offline Reinforcement Learning

Dikshant Gupta
active_cost project report

Abstract

This report documents the `active_cost` project, a research codebase for augmenting offline reinforcement learning with active data collection under an explicit movement or acquisition budget. The project combines offline policy pretraining, ensemble uncertainty estimation, and several budgeted collection strategies ranging from simple uncertainty ranking to optimal-transport-inspired target selection. The repository supports two main algorithmic backbones: TD3+BC for maze2d and locomotion environments, and Implicit Q-Learning (IQL) for antmaze. The current codebase has evolved from early heuristic collectors to a more instrumented implementation that logs collection cost, replay statistics, transport entropy, and reward-cost summaries. This document explains the motivation, code architecture, implemented methods, datasets, training workflow, and the latest available experimental evidence in the workspace, with particular attention to the April 2026 maze2d runs. The report also records practical lessons from the experiments: medium-quality maze2d settings respond well to active augmentation, method 5 currently offers the best upside on maze2d-large-v1, and more sophisticated planners improve peak performance only when their computational overhead and failure modes are controlled.

1 Introduction

Offline reinforcement learning (RL) aims to learn control policies from fixed datasets without repeated online interaction. This setting is attractive when online trials are expensive, slow, or unsafe, but pure offline training is often limited by dataset support mismatch, sparse coverage around goal states, and compounding extrapolation error [Levine et al., 2020, Fu et al., 2020]. In navigation-style domains such as antmaze and maze2d, these issues are especially visible: a static dataset may contain suboptimal or geographically skewed trajectories, while the states that matter most for improved control can be separated from the current data manifold by long movement paths or low-density regions.

The `active_cost` project explores a middle ground between purely offline RL and unrestricted online RL. The central idea is to begin from an offline-trained policy and then actively collect a limited number of additional transitions under a fixed per-epoch budget. The budget is interpreted as a movement or retargeting cost: selecting a promising state to visit is useful only if the policy can reach it without consuming too much of the collection allowance. This framing turns active data collection into a sequential resource-allocation problem. A good collector must identify novel or informative states, reason about support and reachability, and avoid spending the budget on targets that are either too expensive or too redundant.

The codebase implements several collection strategies with increasing sophistication. The simplest methods rank candidates by uncertainty. More advanced variants reason jointly about uncertainty, diversity, and travel cost through optimal-transport (OT) objectives. The current TD3+BC active pipeline also integrates replay reweighting, diagnostic logging, and plotting, which makes the repository suitable both for experimentation and for producing research artifacts.

This report has two goals. First, it serves as a technical record of how the repository is organized and how the implemented methods work. Second, it summarizes what the experiments in the current workspace actually show, rather than describing an aspirational design detached from code and logs. The document therefore focuses on the code paths and results that are present in the repository as of April 7, 2026.

2 Problem Setting

The project studies active augmentation of offline RL datasets. Let $\mathcal{D}_0$ denote an initial offline dataset, typically loaded from D4RL or from pruned subsets stored in the local `DATASETS/` directory. An offline RL algorithm first fits a policy π_0 on $\mathcal{D}_0$. Then, for each active epoch $e \in \{1, \dots, E\}$, the agent is given a budget B_e and may interact with the environment to add up to N new samples to the replay buffer. The active

procedure selects targets in state space, moves toward them, rolls out short trajectories, and appends the collected transitions to the training data. After collection, the policy and uncertainty model are updated.

The system is designed around the following tradeoffs:

1. **Information versus cost.** Highly uncertain states may be informative, but they may also be expensive to reach from the current state.
2. **Novelty versus support.** Leaving the data manifold can improve coverage, but it also increases the risk of collecting implausible or low-value samples.
3. **Immediate gain versus long-horizon structure.** Greedy target selection may miss useful frontier expansion opportunities that only become valuable after multiple retargeting steps.
4. **Sample quality versus runtime.** More elaborate planners can improve peak reward while significantly increasing wall-clock time.

These tradeoffs are reflected in the method implementations and in the logs from the larger sweeps. Methods 1 and 3 are comparatively robust and simple. Method 5, which uses OT-based selection and richer diagnostics, often performs better on maze2d but is computationally heavier and less stable in some broader runs.

3 Repository Overview

The repository is organized around a small set of training scripts, utilities, datasets, checkpoints, and experiment logs.

3.1 Core training scripts

- `td3bc_offline.py`: offline TD3+BC training for maze2d and locomotion settings.
- `td3bc_active.py`: active TD3+BC training with uncertainty models, cost-aware collection, replay reweighting, and diagnostics.
- `iql_offline.py`: offline IQL training for antmaze.
- `iql_active.py`: active IQL training for antmaze with a related collection interface.

3.2 Utility modules

- `utils_common.py`: shared environment constants, seeding, reward utilities, logging helpers, and compatibility wrappers.
- `utils_uncertainty.py`: ensemble dynamics model, pairwise cost/value regressors, OT planner, and the main `Collector` implementation.
- `utils_plotting.py`: collection visualizations, uncertainty plots, reward and cost curves, and OT decision plots.
- `utils_dataset.py`: dataset pruning and conversion helpers.

3.3 Artifacts and experiment folders

- `TD3BC_OFFLINE/` and `IQL_OFFLINE*`: offline checkpoints.
- `TD3BC_FINAL/` and `IQL_ACTIVE/`: active run outputs, per-epoch logs, plots, and final models.
- `logs/`: launcher logs and summary tables for batch experiments.
- `DATASETS/`: local HDF5 datasets and pruned subsets.

The workflow is intentionally simple: shell launchers enumerate environment-dataset pairs, check for offline checkpoints, and invoke the appropriate active script with method-specific flags. This makes experiments easy to resume and inspect, but it also means that naming collisions and partially completed runs must be interpreted carefully when summarizing results.

4 Offline RL Backbone

4.1 TD3+BC

For maze2d and locomotion tasks, the repository uses TD3+BC [Fujimoto and Gu, 2021]. The offline script in `td3bc_offline.py` loads either the canonical D4RL dataset or a local pruned dataset, fits a standardized state scaler, trains a `d3rlpy` TD3+BC model, evaluates it periodically on the environment, and saves both the policy and an uncertainty ensemble. The important implementation point is that offline training is treated as a bootstrap stage rather than the final objective. The active script later injects the saved model parameters into a fresh policy instance and continues training after each collection epoch.

TD3+BC is a reasonable backbone for the maze2d experiments because the action spaces are low-dimensional and the method is relatively stable under offline data conditions. The logs from April 2026 confirm that policy updates themselves are not the dominant runtime bottleneck; the heavy cost comes from active selection and collection.

4.2 IQL

For antmaze, the repository uses IQL [Kostrikov et al., 2022]. The active IQL script supports a similar collection interface but uses an IQL policy and, when desired, sparse reward relabeling around inferred navigation goals. Relative to TD3+BC, the IQL side of the repository was used for broader March sweeps across antmaze subsets. Those sweeps are valuable mainly for stability analysis: they reveal that simple active methods complete reliably, while more elaborate variants fail more often across a heterogeneous benchmark.

5 Uncertainty Modeling

The project uses an ensemble uncertainty model implemented in `utils_uncertainty.py`. At a high level, the model predicts next-state dynamics from (s, a) pairs using multiple independently trained networks. Disagreement among these models serves as a proxy for epistemic uncertainty. This uncertainty enters the collector in several ways:

1. ranking candidate target states,
2. calibrating active selection intensity,
3. identifying frontier states for exploration,
4. defining one component of the OT utility score.

The codebase has evolved from a relatively narrow uncertainty-only design into a more modular structure. In addition to the original ensemble dynamics model, the file now includes a pairwise cost regressor and a state-value regressor that can support richer planning heuristics. Not every feature is equally mature in the latest runs, but the direction is clear: active collection is being reframed as a structured decision problem rather than a one-shot uncertainty sort.

6 Relation to Prior Work

This repository sits at the intersection of four strands of literature: offline RL, active or online data acquisition, uncertainty-aware exploration, and optimal-transport-based planning.

6.1 Offline RL

The offline RL backbone follows mainstream practice. TD3+BC [Fujimoto and Gu, 2021] offers a simple and effective way to regularize policy learning toward the behavior data while still exploiting value estimates. IQL [Kostrikov et al., 2022] is particularly well matched to sparse-reward antmaze tasks because it avoids explicit out-of-distribution action maximization in the policy improvement step. The project does not attempt to improve these algorithms in the purely offline phase. Instead, it treats them as robust foundations from which active augmentation can start.

6.2 Uncertainty-guided exploration

The earliest active methods in the repository align with a long tradition of uncertainty-guided exploration in model-based RL and ensemble methods. The key hypothesis is that epistemic uncertainty is a useful heuristic for where new data would be most informative. In pure online RL this often translates into optimistic exploration. In this project the same signal is repurposed for *budgeted acquisition*: the question is not merely whether a state is uncertain, but whether it is uncertain enough to justify the travel cost needed to reach it.

6.3 Optimal transport and distribution repair

The more recent methods, especially method 5, are conceptually closer to OT-based distribution matching [Villani, 2009, Peyré and Cuturi, 2019]. Rather than treating candidate targets independently, the method constructs a soft distribution over candidates and then computes a transport plan from source states to those targets. This is appealing in active data collection because it allows one to represent the collector’s objective at the level of distributions: source states summarize where the agent currently is or knows how to reach, while target weights summarize where the collector wants coverage to improve.

The resulting procedure is not a textbook OT optimization over policy occupancy measures. It is a practical heuristic that borrows OT machinery to balance uncertainty, support, diversity, and travel cost. That distinction matters. The code is best understood as an *OT-inspired active selector* rather than a theoretically closed-form OT solution to a control problem.

6.4 Cost-aware data acquisition

Many active learning systems assume that each label or sample has roughly equal cost. This repository explicitly rejects that assumption in navigation environments. Moving from the current state to a target state can consume a large fraction of the epoch budget, so acquisition cost must be modeled directly. This is one of the strongest conceptual points of the project: once acquisition cost is asymmetric and state-dependent, active data collection becomes a planning problem rather than a ranking problem.

7 Implementation Details of the Current TD3+BC Pipeline

Although the high-level workflow is simple, several implementation choices are critical to reproducing the latest behavior of the repository.

7.1 Offline checkpoint injection

The active TD3+BC script does not merely reload a prebuilt trainer. It recreates the policy object, builds it with the active dataset, and then manually injects weights from the saved offline checkpoint through `d3rlpy.torch_utility.set_state_dict`. The same script conditionally reloads the uncertainty ensemble if the offline uncertainty checkpoints are present. This design allows the active script to control all subsequent logging, directory naming, replay handling, and plotting while still starting from the offline solution.

7.2 Device handling

The active script contains explicit device handling because `d3rlpy` and the collector logic mix library-managed GPU state with custom PyTorch modules. After constructing the policy with `use_gpu`, the script

manually overrides internal device pointers when a GPU is available. This is a pragmatic workaround rather than a clean abstraction, but it is one reason the runs can allocate GPU memory correctly while still relying on extensive CPU-side collection logic.

7.3 Replay reweighting

The repository’s recent TD3+BC runs do not train on the expanded dataset uniformly. Instead, they rebuild the training pool each epoch with a return-weighted prioritized sampling scheme. Concretely, episode returns are standardized, exponentiated, clipped to a safe interval, normalized into probabilities, and then mixed with a uniform replay component. This creates a form of soft curriculum: newly collected and higher-return trajectories are emphasized without completely discarding lower-return support.

This mechanism is easy to overlook when interpreting results, but it matters. Method comparisons in the current pipeline reflect *both* the collection strategy and the replay strategy. In other words, method 5’s gains on medium maze2d are not purely due to target selection; they also benefit from a training loop that increasingly favors the better data gathered so far.

7.4 Diagnostics as first-class outputs

The active script logs much more than reward. For each epoch it records reward, instantaneous cost, cumulative cost, replay statistics, and a variety of collector diagnostics. The diagnostic CSV written for method 5 includes budget use, target fractions, target movement cost, uncertainty and diversity summaries, transport entropy, and support-related terms. This instrumentation is not cosmetic. It is what makes it possible to interpret failures and to reason about how the collector is behaving internally.

8 Detailed View of the Collector

The `Collector` class in `utils_uncertainty.py` is the operational core of active data acquisition. It maintains references to the environment, the current policy, the uncertainty model, budget settings, and method-specific hyperparameters.

8.1 Candidate generation

The collector begins from the current state pool, which in navigation settings is often just the set of observations already present in the dataset. Candidate generation is not purely uniform. The recent method 5 implementation uses frontier-biased sampling to focus planning on regions that may repair coverage rather than simply resampling dense interior regions of the buffer. This is an important shift away from naive uncertainty ranking.

8.2 Feasibility and support

Support awareness enters the collector through multiple paths. Some helper functions estimate feasibility through nearest-neighbor structure relative to offline reference observations. Other terms penalize targets that are too far from known support or too costly to route to under the remaining budget. The code’s terminology mixes support, feasibility, and travel, but the conceptual role is stable: these terms are meant to prevent the collector from wasting budget on states that are novel in the wrong way.

8.3 Route execution

The target chosen by the active selector is not always executed in a one-shot manner. In the richer collector paths, the method can plan over a top-mass subset of candidate states and execute several steps of a route under a receding-horizon logic. The current report focuses on the method 5 path because that is what the latest maze2d evidence uses, but the broader collector structure is more general than a single-state chooser.

8.4 Data acquisition semantics

Once a target is selected, the collector computes the movement cost from the current state to the target. If the cost would exceed the remaining budget, the epoch terminates. Otherwise, the collector moves into a short trajectory collection phase, records the resulting transitions into a replay buffer, updates counters, and logs the event. This means that every epoch interleaves discrete target selection and continuous trajectory rollout.

9 Experimental Protocol and Interpretation

9.1 What is actually being compared

The experiments compare active methods after offline pretraining, not from scratch and not against an unconstrained online baseline. Each method begins with the same offline checkpoint for a given environment and split, then repeatedly augments the dataset under a fixed per-epoch budget. A fair reading of the results is therefore: given a fixed offline initialization and a fixed amount of active collection, which target-selection strategy produces the best downstream policy?

9.2 Role of the dataset splits

The **hard** and **medium** labels are easy to misread as generic quality levels. In this repository they are really support conditions. For maze2d-large, the split boundaries are geometric conditions over the y coordinate. A **hard** split does not simply contain lower return trajectories; it also places the policy in a weaker geometric relationship to the goal and to parts of the maze that matter for successful navigation. This explains why the active methods separate more strongly on **hard**: the collector is doing a harder coverage-repair job.

9.3 Metrics

The codebase tracks several metrics that should be treated separately:

- **Final reward**: what the latest completed epoch achieved.
- **Best reward**: the maximum epoch-end reward, useful for measuring upside.
- **Last-five average**: a rough measure of late-epoch stability.
- **Reward-cost AUC**: a cost-aware summary already computed by the script, though not yet aggregated broadly in the logs.
- **Completion rate**: especially important in broad sweeps.

Different methods lead on different metrics. Method 1 currently leads on stable final performance. Method 5 currently leads on peak medium-maze reward. The March IQL sweep shows that completion rate can tell a different story than peak reward.

10 Collection Methods

10.1 Method 1: baseline active selection

Method 1 is the simplest active collector used in the recent sweeps. In practice it serves as the stability baseline. It samples candidate states from the existing state pool, scores them with uncertainty-related heuristics, and collects short trajectories under the epoch budget. Because it avoids the heavier OT machinery, it is comparatively robust and easier to scale across many datasets.

10.2 Method 3: sequence- or diversity-aware heuristic selection

Method 3 adds more structure than method 1, historically through sequence-aware or combinatorial selection heuristics. In the current repository it remains less complex than the OT-heavy methods but can exploit more planning structure than a purely greedy uncertainty rule. The latest maze2d-medium result shows that method 3 can be competitive in terms of final reward, but the hard split results indicate that it is not uniformly stronger than method 1.

10.3 Method 5: OT-based target selection

Method 5 is the most important advanced method in the current TD3+BC pipeline. It uses an OT-inspired planner that combines three ingredients:

$$\text{score}(s_j) = u_j + \delta d_j - \lambda f_j, \quad (1)$$

where u_j is an uncertainty term, d_j is a diversity or novelty term, and f_j is a feasibility or support-related term. These utilities are normalized and converted into a target distribution over candidate states. A transport plan from source states to candidate targets is then computed with entropic regularization, yielding a target mass distribution that is used to sample or select collection targets.

Method 5 also introduces several practical mechanisms that matter in the experiments:

- frontier-biased candidate subsets,
- pruning by movement cost under the remaining budget,
- active/random/initial target mixture tracking,
- per-epoch diagnostics such as transport entropy and selected uncertainty,
- reward-cost summaries after each active epoch.

This method is the best expression of the project’s core thesis: active data collection should not be driven by uncertainty alone, but by uncertainty filtered through feasibility, travel cost, and coverage repair.

10.4 Methods 6–8 and the broader search space

The code shows evidence of experimentation beyond methods 1, 3, and 5. Method 6 continues the OT line with a more explicit two-phase exploration and exploitation policy. Methods 7 and 8 incorporate more ambitious Schrödinger-bridge or CASPI-style ideas. However, the current active TD3+BC script explicitly restricts supported methods to $\{1, 3, 5\}$, and the March mixed sweep indicates that the more sophisticated antmaze variants were substantially less robust. This is an important research lesson: richer planners can be appealing conceptually while still being operationally worse than simpler baselines.

11 Budgeted Active Training Loop

The active training loop in `td3bc_active.py` proceeds as follows:

1. load the offline dataset and pretrained policy,
2. fit or restore a state scaler and uncertainty model,
3. initialize a collector with the current environment, policy, and uncertainty model,
4. for each active epoch:
 - (a) reset the per-epoch budget,
 - (b) visualize uncertainty if the task is navigation,
 - (c) collect a new batch under the chosen method,

- (d) append collected data to the replay dataset,
 - (e) optionally visualize collection and buffer evolution,
 - (f) rebuild the training pool with return-weighted replay,
 - (g) fit the policy for a fixed number of gradient steps,
 - (h) log reward, cost, reward-cost AUC, replay statistics, and collector diagnostics,
 - (i) retrain the uncertainty model on the expanded replay buffer.
5. save the final policy and uncertainty model.

Two design choices are especially notable.

First, replay is reweighted by episode return through `build_return_weighted_episode_pool`. This creates a 70/30 prioritized-versus-uniform mix. The intent is to bias training toward higher-value trajectories without fully collapsing support.

Second, the collector logs diagnostics to `diagnostics_epoch.csv`. This file records quantities such as budget used, transport entropy, active/random/initial target fractions, and selected uncertainty statistics. This instrumentation is valuable for turning the repository into a research platform rather than a black-box launcher.

12 Datasets and Environment Families

The project spans three families of environments:

- **antmaze**: sparse-reward navigation tasks trained mainly with IQL,
- **maze2d**: lower-dimensional navigation tasks trained mainly with TD3+BC,
- **locomotion and kitchen**: non-navigation tasks with TD3+BC support, though the latest evidence in this workspace centers on maze2d and antmaze.

The repository also includes pruned datasets such as **easy**, **medium**, **hard**, **top30**, and **bottom30**. In navigation environments these pruned subsets correspond to geometric filters over state positions. For instance, the maze2d-large splits are defined by vertical thresholds in `utils_common.py`. This means that the data quality label is not simply a reward percentile; it is a spatial support condition that changes which parts of the maze are represented offline.

This detail matters for interpreting the results. The active collector is often trying to repair spatial coverage. Therefore, a method that works well on a **medium** split may be exploiting a moderate support gap, while a method on a **hard** split may be facing a much more severe reachability bottleneck.

13 Experiment Infrastructure

13.1 Batch launchers

The repository provides multiple shell launchers. Two are central:

- `run_all_methods_all_datasets_mixed.sh`: broad mixed sweeps across algorithms and dataset families.
- `run_maze2d_existing_offline_m1_m3_m5.sh`: focused TD3+BC sweep on maze2d using existing offline checkpoints.

These scripts write a launcher log, a run log, and a tab-separated summary. The summaries record command lines and statuses, which makes them useful for reconstructing what actually finished. The downside is that some run directories do not encode the dataset split in their folder names, so post hoc attribution occasionally requires combining summary files, timestamps, and launcher ordering.

13.2 Logging and diagnostics

The active scripts produce several levels of logging:

- per-step training CSV files from `d3rlpy`,
- high-level batch summaries under `logs/`,
- per-epoch diagnostic CSVs for active collection,
- visualizations of uncertainty maps, collections, and reward curves,
- optional Weights & Biases or TensorBoard logging hooks.

This is a strength of the project. Many research codebases expose only aggregate reward curves. Here, the logs contain enough detail to diagnose why a run is slow, whether it is GPU-bound, how much budget is being spent, and how the collector is mixing target types.

14 Experimental Evidence in the Current Workspace

The most reliable evidence in the workspace comes from two sources:

1. the March 18, 2026 mixed IQL sweep under `logs/all_methods_all_datasets_mixed/`,
2. the April 6–7, 2026 maze2d TD3+BC sweep under `logs/maze2d_existing_offline_m1_m3_m5/`.

14.1 Mixed March sweep: stability first

The March mixed sweep is most informative about robustness. On the IQL side, methods 1 and 3 completed all 27/27 active jobs recorded in the summary. Method 5 completed only 12/26, while methods 7 and 8 completed 10/26 and 11/26, respectively. The implication is straightforward: more elaborate collection logic was not yet production-ready across heterogeneous antmaze-style tasks.

This result should not be dismissed as mere engineering friction. Stability is itself a scientific outcome in budgeted active RL. A method that occasionally peaks higher but fails on half the benchmark cannot be treated as a superior default collector.

14.2 April maze2d sweep: reward comparison

The April maze2d run is the clearest current benchmark for TD3+BC-based active collection. The run order in `run_maze2d_existing_offline_m1_m3_m5.sh` and the artifact timestamps show that the latest completed and in-progress runs correspond to `maze2d-large-v1` on the `hard` and `medium` splits.

Table 1: Latest maze2d-large-v1 active results available in the workspace. “Best” is the best epoch-end reward observed so far in the run. “Last5” is the mean of the final five completed epochs. Method 5 on `medium` is still running at the time of this report.

Split	Method	Final	Best	Mean	Last5
hard	1	445.30	467.40	221.93	454.49
hard	3	218.80	218.80	54.22	147.35
hard	5	252.25	421.40	157.77	336.39
medium	1	559.10	559.10	346.28	525.05
medium	3	519.60	571.30	301.29	492.70
medium	5 (in progress)	511.55	618.25	383.70	545.93

Several patterns stand out:

1. **Medium is consistently easier than hard.** All three methods perform substantially better on `medium` than on `hard`. This aligns with the spatial-pruning interpretation of the dataset splits.

2. **Method 1 is the strongest completed method by final reward.** It wins the completed `hard` and `medium` comparisons in final reward.
3. **Method 5 has the best upside on medium.** Even before completion, it has achieved the highest peak reward in the latest `medium` run, at 618.25.
4. **Method 3 is competitive on medium but weak on hard.** Its `medium` run is strong, but its `hard` result is poor relative to methods 1 and 5.

Taken together, the latest `maze2d` evidence suggests the following practical ranking:

- If reliability and final completed performance matter most, method 1 is the current default.
- If peak performance on `maze2d-medium` matters most and runtime is acceptable, method 5 is the most promising.
- Method 3 occupies a middle ground but does not currently dominate either objective.

14.3 Method 5 diagnostics

The latest `medium` method 5 run includes per-epoch diagnostics. Over the first 13 completed epochs, the average transport entropy is approximately 2.919, with values very close to 3.0 in the logged epochs. This indicates that the transport mass is not collapsing onto a single target; the OT distribution remains fairly spread over candidate targets. The same run also records active, random, and initial target fractions each epoch, showing that the policy is using a mixed targeting scheme rather than relying exclusively on active picks.

This is encouraging from a design perspective. It suggests that the collector is not simply acting as a deterministic greedy selector. Instead, it maintains some target diversity while still producing strong reward peaks.

15 Runtime Behavior and Systems Findings

One practical question that arose during the latest run was whether the method was actually using a GPU and why it still seemed slow. The answer is visible from both code and logs.

The live April 7 run was indeed on GPU 0: `nvidia-smi` showed the active Python process occupying roughly 16 GiB of GPU memory. However, instantaneous GPU utilization was low. This is not a contradiction. The run is slow because the dominant wall-clock cost is not the TD3+BC gradient update itself. Instead, time is spent in the active collection loop:

- repeated environment stepping,
- uncertainty scoring,
- candidate pruning and frontier sampling,
- OT plan construction,
- plotting and diagnostics,
- retraining the uncertainty model after each epoch.

The training metrics in the log show that the actual algorithm update cost is only a few milliseconds per step. The slow part is the sequential CPU-heavy collection and planning logic. This matters for future work: any next iteration of the project should treat runtime optimization as a first-class research concern, not merely a post-processing task.

15.1 Why GPU memory can be high while utilization is low

This point is worth making explicitly because it can otherwise look like a misconfiguration. The policy and uncertainty modules are resident on the GPU, so model parameters, activations, and temporary tensors can occupy substantial memory. However, the program alternates between short GPU computations and much longer CPU-side work such as environment interaction, candidate processing, Python control flow, CSV writing, and plotting. As a result, the GPU remains allocated but often underutilized. This is consistent with the observed `nvidia-smi` snapshot: high memory usage, low utilization, and an actively advancing log file.

15.2 Implications for benchmark design

The runtime profile suggests a practical guideline for future sweeps. When evaluating a new collection method, one should first profile wall-clock time on a narrow benchmark such as `maze2d-large-v1` before launching a broad all-datasets sweep. Otherwise the comparison will mix algorithm quality and implementation inefficiency in a way that is difficult to untangle. The March and April runs already demonstrate this hazard.

16 Strengths of the Current System

The project has several concrete strengths.

16.1 Strong experimental instrumentation

The repository has become far more inspectable than a typical prototype. Reward CSVs, diagnostics, plots, summary TSVs, and per-epoch directories make it possible to reconstruct experimental state with relatively little ambiguity.

16.2 Clear progression from simple to advanced methods

The collector family is organized as a progression from simple uncertainty heuristics to richer OT-based planning. This is a sensible research strategy because it allows direct comparison between operationally stable baselines and higher-upside methods.

16.3 Separation of offline pretraining and active augmentation

The scripts treat offline RL as a reusable initialization stage. This is computationally efficient and scientifically useful because it isolates the value of the active augmentation phase.

16.4 Task-specific backbone choice

Using IQL for `antmaze` and TD3+BC for `maze2d` is a practical choice. It avoids forcing a single backbone across very different domains and leverages the strengths of each algorithm family.

17 Current Limitations

17.1 Naming and result attribution

Some active run directories omit the dataset split in their names. As a result, reconstructing which run corresponds to `hard` versus `medium` requires combining timestamps and launcher ordering. This should be fixed in future experiment naming.

17.2 Runtime overhead

Method 5 is promising but slow. The collector performs extensive CPU-side work every epoch and retrains the uncertainty model for many steps. Without profiling and pruning, large sweeps will remain expensive.

17.3 Method maturity is uneven

The code contains multiple ambitious planning variants, but only a subset are currently robust enough to use as default methods. The mixed March sweep makes this explicit.

17.4 Evaluation breadth is incomplete on the TD3+BC side

The broad April 6 TD3+BC all-datasets sweep did not meaningfully progress in the recorded logs, so the strongest current results are still concentrated on maze2d rather than covering all supported environment families.

17.5 Theoretical framing lags implementation

The project’s implementation has become richer than its current explicit theory. Method 5 is motivated by transport, coverage repair, and support-aware exploration, but the code does not yet expose a clean formal objective that would let one derive the method from first principles. This is not unusual for a research codebase, but it means that the current paper should be positioned as an empirical systems-and-methods report rather than as a theorem-driven algorithm paper.

17.6 Limited seed averaging

The latest strongest evidence in the workspace is largely from single-seed runs or launcher summaries. That is enough to identify promising directions, but not enough to make strong claims about variance. A publication-quality benchmark would require multiple seeds and a more standardized aggregation pipeline.

18 Recommendations for Next Iteration

Based on the current code and experiments, the next round of work should prioritize the following.

18.1 Make method 5 the main experimental branch, but optimize it

Method 5 currently offers the strongest upside in maze2d-medium. The best investment is not inventing an even more complex planner immediately, but making method 5 cheaper and more reliable. Candidate opportunities include:

- reducing per-epoch uncertainty retraining,
- caching cost and feasibility computations,
- lowering plotting frequency during sweeps,
- profiling collector hotspots and vectorizing more of the OT-related path.

18.2 Preserve method 1 as the control baseline

Method 1 should remain the baseline in every sweep. Its stability makes it the proper control when testing new collector ideas.

18.3 Improve run naming and metadata

Each experiment directory should encode environment, split, method, seed, and perhaps a short configuration hash. This would remove ambiguity in later analysis.

18.4 Expand reward-cost analysis

The active TD3+BC script already computes reward-cost AUC quantities. The next step is to aggregate these across runs and compare methods not only by final reward but by reward gained per unit of budget consumed.

18.5 Separate algorithmic novelty from systems overhead

Future ablations should isolate the contribution of each major component of method 5:

- OT transport itself,
- frontier-biased candidate sampling,
- support or feasibility penalties,
- replay prioritization,
- uncertainty retraining frequency.

Without such ablations, it is easy to attribute all gains to the OT formulation when some fraction may come from replay reweighting or from better frontier sampling alone.

18.6 Create a reproducible reporting pipeline

The repository already contains enough raw artifacts to support automated reporting. A lightweight analysis script should ingest the summary TSVs and per-run reward CSVs, identify the canonical latest run for each environment-split-method tuple, and emit tables directly into LaTeX. That would eliminate manual timestamp-based reconstruction and reduce the risk of reporting errors.

18.7 Resume the all-datasets TD3+BC sweep only after narrowing configuration risk

The broad sweep should be resumed after confirming a narrower set of well-performing settings. The current evidence argues against launching expensive large-scale grids before the collector runtime and method defaults are tightened.

19 Conclusion

The `active_cost` project is a substantial research codebase for studying how offline RL can be improved by carefully budgeted active data collection. Its central contribution is not a single final algorithm, but an experimental framework that combines offline pretraining, uncertainty estimation, cost-aware selection, and rich diagnostics. Within that framework, the latest evidence supports three main conclusions.

First, simple methods still matter. Methods 1 and 3 have been the most reliable across broad sweeps, and method 1 is currently the best completed performer on the latest maze2d-large-v1 runs.

Second, OT-based active collection is promising. Method 5 achieves the best upside in the current maze2d-medium experiments and captures the intended balance among uncertainty, diversity, and travel cost more faithfully than simpler heuristics.

Third, systems concerns are part of the science. The active collector is currently slow because the expensive part of the pipeline is sequential collection and planning, not GPU training. This affects which methods are practical to evaluate and how broadly they can be benchmarked.

Overall, the project is beyond the stage of a one-off prototype. It already contains enough infrastructure, instrumentation, and empirical signal to support a real paper cycle. The next phase should narrow the method set, optimize the strongest candidate, and convert the current per-run evidence into cleaner aggregate comparisons across tasks and seeds.

Broader Perspective

From a broader RL research perspective, the repository illustrates a useful methodological point. Offline RL is often framed as the endpoint of a pipeline: one receives a fixed dataset and must do the best possible job with it. This project instead treats offline RL as the *starting point* for controlled, selective interaction. That shift is valuable because many real systems are not purely offline or purely online. They have some historical data, some limited ability to gather more, and a real cost for doing so. Methods that can reason about where to spend that limited interaction budget are therefore likely to matter in practice even if they do not fit neatly into the standard offline-online dichotomy.

References

- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Scott Fujimoto and Shixiang Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.
- Cédric Villani. *Optimal Transport: Old and New*. Springer, 2009.

A Appendix: Reproducibility Notes

The following details are relevant for reproducing the current report from the repository state.

- Date of report preparation: April 7, 2026.
- Repository working directory: `/data/home/dikshantg/active_cost`.
- Main active maze2d launcher: `run_maze2d_existing_offline_m1_m3_m5.sh`.
- Main mixed sweep launcher: `run_all_methods_all_datasets_mixed.sh`.
- Core latest summary files:
 - `logs/maze2d_existing_offline_m1_m3_m5/summary_20260406_202453.tsv`
 - `logs/all_methods_all_datasets_mixed/summary_20260318_152008.tsv`
- Current live run at report time: `maze2d-large-v1`, `medium`, method 5 in tmux session `maze2d_existing_20260406`.

B Appendix: Important Files

- `td3bc_active.py`: main active TD3+BC loop, replay reweighting, diagnostics, and uncertainty retraining.
- `utils_uncertainty.py`: collector logic, OT planner, ensemble uncertainty, and auxiliary regressors.
- `run_maze2d_existing_offline_m1_m3_m5.sh`: focused April 2026 maze2d launcher.
- `logs/maze2d_existing_offline_m1_m3_m5/summary_20260406_202453.tsv`: authoritative status table for the current maze2d batch.
- `TD3BC_FINAL/maze2d-large-v1_OT/...`: per-run reward CSVs, plots, and final checkpoints.

C Appendix: Interpreting the Latest Live Run

At the time of writing, the latest live run is the `maze2d-large-v1`, `medium`, method 5 active TD3+BC job launched through tmux on April 6, 2026. The process is still active, and the run has already completed 14 active epochs. This report therefore treats the method 5 medium result as provisional, reporting the best and latest completed values rather than claiming a finished endpoint.

D Appendix: Suggested Paper Positioning

If this report is converted into a conference submission, the most defensible framing is:

1. introduce active budgeted data collection on top of offline RL,
2. present uncertainty-only and OT-inspired collectors as a progression,
3. emphasize reward-cost tradeoffs and completion robustness,
4. focus on the empirical lesson that support-aware transport heuristics can improve peak performance on navigation tasks while introducing nontrivial systems overhead.

This positioning is stronger than claiming a universally superior algorithm. The current evidence supports a nuanced claim: transport-based collectors are promising, but their value emerges only when measured alongside cost, runtime, and operational reliability.